PYTHON

REVISION NOTEBOOK

Complete chapter-wise notes • Chapters 1–54

A study-friendly rewrite of the supplied Python course transcript

READ • TRACE • PRACTISE • REVISE

Keep this notebook beside your editor.
Write the examples by hand, run them, and explain each output.

PYTHON REVISION NOTEBOOK

Complete chapter-wise handwritten-style notes from the supplied transcript

Purpose: Learn, revise, and practise the full course in the same order as the transcript.
Coverage: Chapters 1-54 | Python basics, collections, functions, control flow, OOP, exceptions, and multithreading.

> **How to use this notebook**
>
> Read the **Core idea** first. 2. Re-type the **code to remember** examples instead of only reading them. 3. Trace the outputs by hand. 4. Finish each chapter with the **Quick recall** prompts. 5. Return to the final revision sheets before an interview or assessment.

Transcript note: This notebook preserves the chapter order, concepts, explanations, and useful demonstrations from the supplied transcript. Repeated spoken phrases and incidental commentary are shortened. Clear speech-to-text errors in technical names and Python syntax are normalized for readability (for example, **Jupyter**, `__name__`, and `tolist()`). Course-specific examples and the instructor's framing are retained where useful.

CONTENT MAP

1. Introduction to Python
2. First Code in Python
3. Python Setup
4. IDE Installation
5. Python IDLE and Exploring Number Operations
6. Working with Strings
7. More on Strings
8. List in Python
9. Tuple in Python
10. Set in Python
11. Dictionary in Python
12. Variable Storage in Python
13. Data Types in Python
14. Operators in Python
15. Introduction to Function
16. Working with Module with `math`
17. Swapping of Variables in Python
18. User Input
19. `if` Statement
20. `else` Statement and Debugging
21. Nested `if`
22. `elif` Statement
23. `match` Statement
24. `while` Loop
25. `for` Loop
26. `break` and `continue`
27. Array
28. Array Functions
29. Arguments in Function
30. Global vs Local Variables
31. Factorial in Python
32. Recursion
33. Factorial Using Recursion
34. Higher-Order Function
35. Anonymous Function Using `lambda`
36. `filter()` Function
37. `map()` and `reduce()` Functions

38. Inner Function
39. Decorators in Python
40. Modules and Packages
41. Special Variable `__name__`
42. OOP Introduction
43. Class and Object
44. `__init__()` Method
45. Constructor Using `__new__()` Method
46. Types of Methods
47. Inheritance and MRO
48. `__init__()` and `super()` with Inheritance
49. Polymorphism and Duck Typing
50. Operator Overloading
51. Method Overriding
52. Abstraction in Python
53. Exceptions in Python
54. Multithreading in Python

PART A - PYTHON FOUNDATIONS

CHAPTER 1 - INTRODUCTION TO PYTHON

Core idea

Python is a **high-level, general-purpose, multi-paradigm programming language**. The transcript states that Python was first released in **1991**. It is used to build many kinds of applications rather than being limited to one task.

What each description means

- **High-level language:** Its syntax is closer to human-readable English than to machine instructions. The computer ultimately works with binary (0s and 1s), so Python code must be translated/executed by language software such as an interpreter (the transcript also introduces the broader compiler/interpreter idea).
- **General-purpose:** Python can be used for desktop utilities such as calculators, web applications, AI-based applications, scientific/numeric work, software development, and system administration.
- **Multi-paradigm:** Python supports more than one programming style:
  - Procedure-oriented: organize work around procedures/steps (the transcript uses C as an example).
  - Object-oriented: organize work around objects (Java is used as an example).
  - Functional: organize work around functions (JavaScript is mentioned as an example).
  - Structured programming is also included in the transcript's description of Python.

Why Python is popular in the transcript

55. It is comparatively easy to read and learn.
56. Its creator, **Guido van Rossum**, wanted programming to be accessible to more people.
57. It has a large community.
58. It has a large collection of libraries for different domains; the transcript especially highlights AI libraries.
59. It is widely used in AI and web development.

Areas and examples named

- Web development: **Django**, **Flask**, **Tornado**.
- Scientific and numerical computing: **SciPy** and **Pandas**.
- AI / machine-learning applications.
- General software development and system administration.

Key mental model

Python source code -> interpreter/compiler implementation -> machine-executable instructions -> program output

Quick recall

- Define high-level, general-purpose, and multi-paradigm.
- Why does a machine need code to be translated?
- Name the web frameworks and scientific libraries mentioned in the transcript.

CHAPTER 2 - FIRST CODE IN PYTHON

Core idea

To run a program, we need (1) a place to write the code and (2) a way to execute it. The transcript first uses an online compiler so that Python can be tried without installing anything.

The online compiler demonstration

- An online compiler such as the one shown in the transcript can run multiple languages (C, Java, VB, and Python were demonstrated).
- The C and Java versions of "Hello World" require more visible setup code than the Python example.
- The Python option shown is **Python 3**. Python 2 is described in the recording as end-of-life; the course uses Python 3.
- The top portion of the editor may contain a comment; the executable statement is the code itself.

First Python statement

```
print("Hello World")
```

- `print` is a built-in function used to display output.
- The parentheses contain the value to print.
- Click **Run** in the online environment; the code runs on the provider's server and shows the output.
- A terminal/black output window is normal. The important point is the output, not the colour or style of the window.

How the parts fit together

- The IT industry stores, fetches, and processes data.
- Software performs these jobs, and software is built with code.
- Python code needs to be executed by an interpreter/runtime.
- A Python backend may serve a web application while frontend work can use HTML, CSS, and JavaScript (as described in the transcript).

Quick recall

- What does `print()` do?
- What is the difference between the code editor and the output/terminal area?
- Why is an online compiler useful before local installation?

CHAPTER 3 - PYTHON SETUP

Core idea

Install Python locally from its **official website** and make sure the terminal can find the interpreter.

Installation checklist

60. Visit the official Python website and open **Downloads**. Avoid unofficial installers because they may be unsafe.
61. Choose the operating system (the demonstration uses Windows).
62. Select the installer matching the machine architecture:
    - Windows 64-bit installer for an x64 Intel/AMD machine.
    - ARM installer for an ARM-based machine.
63. Prefer a **stable release** for a beginner setup. Pre-release/beta versions may contain untested changes and can make troubleshooting harder.
64. Before installing, check whether Python is already available.

Check from the terminal

```
python --version
python3 --version
```

- If the terminal says Python was not found, it is not available through that command/path.
- The transcript checks both `python` and `python3` because the command name can vary by setup.

Important installation option

On the Windows installer screen, select:

`Add python.exe to PATH`

This lets the terminal locate Python without manually configuring the location. Then choose **Install Now** and complete the installer.

Verify after installation

65. Close and **reopen** the terminal after installation so it refreshes the environment/PATH.
66. Run `python --version` again.
67. If `python` does not work, try `python3 --version`.
68. The transcript mentions that an alias can be configured if you prefer one command name over another.

Interactive Python prompt

```
python
```

A Python version banner and the `>>>` prompt indicate that the interactive Python prompt is ready.

```
print("Hello")
```

Type a Python statement at `>>>` and press Enter to execute it immediately.

Run a saved file

- Use an editor such as Notepad to write code.
- Save it in a folder such as `PythonCodes`.
- Move the terminal into that folder, then run:

```
python demo.py
```

- The transcript first tries a `.txt` file and demonstrates that it can still execute, but recommends the `.py` extension because Python tooling, imports, and IDE features expect a Python file.

Remember

- `PATH` connects the terminal command to the Python installation folder.
- Reopen the terminal after changing/installing software.

- Use .py for Python source files.

CHAPTER 4 - IDE INSTALLATION

Why use an IDE?

Writing code is only one part of software development. A development environment can bring together:

- Code editing.
- Running code.
- Debugging (finding and removing bugs).
- Documentation and project organisation.
- Packaging work for deployment (the transcript mentions cloud deployment).

IDE = Integrated Development Environment.

Tools compared in the transcript

Tool	What the transcript highlights	Best fit described
VS Code	Microsoft-built, open-source editor; supports many languages through extensions.	General development and the main editor used in the course.
PyCharm	JetBrains IDE focused on Python; Community and Pro/unified options are discussed.	Python-focused projects and built-in IDE features.
Jupyter Notebook / JupyterLab	Notebook cells can contain code or Markdown notes; output and graphs appear close to the code.	Learning, experiments, short code, Pandas, and plots.

VS Code workflow

69. Download and install VS Code for the operating system.
70. Open a project folder (for example, PythonCodes).
71. Create a file such as demo1.py.
72. Type and save:

```
print("Welcome to TalisCoCo")
```

73. Open **Terminal -> New Terminal** and run:

```
python demo1.py
```

74. The Run/play button can also execute the file when the Python extension and .py extension are available.

VS Code left-side areas

- **Explorer:** project folders and files.
- **Search:** locate text/files.
- **Source Control:** Git-related work.
- **Run and Debug:** execute and debug code.
- **Extensions:** add support for languages and features.

Install the **Python** extension (the transcript describes the Microsoft Python extension as the essential one). Pylance, themes, and other add-ons may appear, but are not all required for the first run. Restart VS Code if requested.

PyCharm workflow

75. Download the unified product or Community option described in the recording.
76. Create a **New Project** and select the Python interpreter/version (the example uses Python 3.13).
77. Right-click the project and choose **New -> Python File**; .py is supplied automatically.
78. Type print("Hello") and use the Run option.

79. The project view, Python console, terminal, and version control are available in the IDE.

The transcript compares PyCharm Pro with the Community edition. It quotes **about $99 for the first year** for an individual Pro licence and says that teachers/students could access it free with an eligible education email. It also describes a unified product that exposes Pro features for an initial one-month period before some features stop. These are details stated in the recording, not a guarantee of current pricing or eligibility.

Jupyter workflow

80. Open Jupyter Notebook in the browser (or install it locally).
81. Select **File -> New Notebook** and choose a Python kernel.
82. Enter code in a cell and press Run:

```
print("Hello")
```

83. Output appears beneath the cell.
84. Use Markdown cells for explanations and notes; the notebook is useful for seeing Pandas output and graphs.

Decision from the lesson

- Use VS Code or PyCharm for larger multi-file projects.
- Use Jupyter for experiments, small code, notes, and data/visual work.
- IDE preference does not change the Python code itself.

CHAPTER 5 - PYTHON IDLE AND EXPLORING NUMBER OPERATIONS

Command Prompt vs PowerShell

Environment	Demonstrations from the transcript
Windows Command Prompt	`dir` lists files; `cls` clears the screen; entering `2 + 3` is rejected because it expects a shell command.
PowerShell	Supports commands such as `dir`/`ls`, `clear`/`cls`, scripting, `echo`, and arithmetic expressions such as `2 + 3`.

PowerShell looks similar to Command Prompt but is a more capable shell; the transcript compares the shell idea to Linux/macOS command environments such as Bash.

IDLE and REPL

- Typing `python` in a terminal opens the interactive Python prompt.
- The transcript calls this Python **IDLE** (Integrated Development and Learning Environment) and introduces the separate IDLE window/editor.
- REPL means **Read - Evaluate - Print Loop**: read the statement, evaluate it, show the result, and repeat.
- The interactive prompt is useful for testing short expressions. Use a file/IDE for longer programs.

Arithmetic operators

Operator	Meaning	Example	Result
`+`	Addition	`5 + 9`	`14`
`-`	Subtraction	`9 - 4`	`5`
`*`	Multiplication	`4 * 3`	`12`
`/`	True division	`8 / 2`	`4.0`
`//`	Floor/quotient division	`9 // 2`	`4`
`%`	Modulo/remainder	`9 % 2`	`1`
`**`	Power/exponent	`8 ** 3`	`512`

Important: ^ is not the exponent operator in Python; the transcript points out that it belongs to bitwise operations. Use ** for powers.

Order of operations (BODMAS)

```
9 + 2 * 3      # 15: multiplication first
(9 + 2) * 3    # 33: brackets first
```

Division: quotient and remainder

For 9 / 2, the result is 4.5 (a floating-point value). The quotient and remainder can be requested separately:

```
9 // 2       # 4
9 % 2        # 1
```

The underscore in the interactive prompt

The prompt's _ refers to the last displayed result. Example sequence:

```
8 ** 3       # 512
5 + _        # 517
2 + _        # 519 (now _ refers to 517)
```

It is convenient for immediate prompt experiments, but use named variables when a value must be retained reliably.

Variables as named references

A variable is introduced as a named "box" that stores/references a value:

```
num1 = 8 ** 3
num2 = 8
num3 = num1 + num2
print(num1)
print(num3)
```

- = assigns a value.
- An assignment statement does not automatically display its result in a script or prompt.
- In a script, use print() to display values.

CHAPTER 6 - WORKING WITH STRINGS

Core idea

Text in programming is usually represented by a **string** (str). A bare word is treated as a variable name; put text inside quotes.

```
num = 8
print(num)        # variable value
print("Naven")    # string literal
print('Naven')    # single quotes also work
```

Why quotes are necessary

- 45 is recognised as a number.
- num works if a variable named num has been assigned.
- A bare name such as Naven is not automatically text; Python tries to find a variable with that name.
- Use 'Naven' or "Naven" to identify it as a string.
- Strings may contain multiple words: 'Naven is ready to go'.

Prompt display vs `print()`

When a string literal is entered directly at the prompt, Python displays its representation, including quotes. `print('Naven')` displays the text itself without the surrounding quote marks.

Quote inside a string

A quote that matches the opening quote can terminate a string early. Use the other quote type or escape the quote with a backslash:

```
message = "Naven's laptop"
message2 = 'He said "Hello"'
message3 = 'Naven\'s "TalisCoCo"'
print(message3)
```

Combining and repeating strings

```
first = "Naven"
last = "TalisCoCo"
print(first + " " + last)   # concatenation
print("Hi! " * 3)           # repetition
```

- `+` adds numbers but concatenates strings.
- Add spaces explicitly when joining words.
- Adjacent string literals can be placed next to each other in source code and are joined by Python; variables need `+` for explicit concatenation.

Quick recall

- What does Python assume when it sees an unquoted identifier?
- How can a single quote be included in a single-quoted string?
- What is the difference between entering a string at `>>>` and using `print()`?

CHAPTER 7 - MORE ON STRINGS

Strings are sequences

A string is an ordered sequence of characters. Each character has an index.

For a string such as:

```
text = "TalisCo"
```

- Positive indexing starts at `0`.
- The final valid index is `len(text) - 1`.
- Negative indexing starts at the end: `-1` is the last character, `-2` the previous one.

```
text[0]      # first character
text[3]      # character at index 3
text[-1]     # last character
text[-2]     # second-last character
```

An index beyond the valid range raises `IndexError`.

Slicing

Syntax:

```
text[start:stop]
```

- `start` is included.

- stop is excluded.
- Omitting start means begin at the start.
- Omitting stop means continue to the end.

```
text[0:3]     # first three characters
text[3:7]     # characters at 3, 4, 5, 6
text[2:]      # index 2 through end
text[:4]      # beginning through index 3
```

A stop index beyond the length is clipped to the end for slicing; this differs from direct indexing.

Immutability

Strings are **immutable**. You can reassign a variable to a new string, but you cannot change one character inside the existing string:

```
name = "Naven"
# name[0] = "K"     # TypeError: string does not support item assignment
name = "Kaven"       # valid: rebind name to another string
```

Useful string operations shown

- len(text) gives the number of characters.
- \n inserts a newline.
- Triple quotes ('''…''' or """…""") allow multiline string text.

CHAPTER 8 - LIST IN PYTHON

Core idea

A **list** stores a collection of values in one variable. It is created with square brackets and is mutable.

```
nums = [45, 87, 21, 24, 99]
names = ["Naven", "Hush", "Kiran"]
mixed = ["Naven", 67, 6.5]
```

A list can contain different data types, and it can also contain other lists (nested lists):

```
mix = [nums, names]
mix[0]      # the nums list
mix[0][0]    # 45
```

The outer list has two elements; nested indexing moves into the inner list.

Indexing and slicing

- Indexing begins at 0; for five values the last index is 4.
- Negative indexing starts at the end.
- Slicing uses list[start:stop] with stop excluded.

```
nums[0]
nums[-1]
nums[2:4]
nums[2:]
```

Lists can be concatenated with + to create a combined list.

Common list methods

Method	What it does

Method	What it does
append(x)	Adds one item at the end.
extend(iterable)	Adds each item from another iterable/list.
insert(i, x)	Inserts x at index i; later items shift right.
remove(x)	Removes the first matching value.
pop(i)	Removes and returns item at index i; pop() removes the last item.
clear()	Removes all items.
copy()	Makes a shallow copy.
count(x)	Counts occurrences of x.
index(x)	Returns the first index of x.
reverse()	Reverses the list in place.
sort()	Sorts the list in place.

Important demonstrations

```
nums.append(77)
nums.insert(1, 50)
nums.remove(24)
last_value = nums.pop()
nums.extend([10, 20, 30])
nums.reverse()
nums.sort()
```

- pop() is compared with a **stack**: last in, first out (LIFO).
- del nums[2:4] deletes a slice; the end index is excluded.
- Slices can also be assigned: nums[2:4] = [54, 76].
- min(nums), max(nums), and sum(nums) are useful built-ins for numeric lists.
- The transcript also demonstrates min() and max() on strings (lexicographic ordering); sum() is for numeric values, not strings.

Remember

Lists are mutable: changing an element changes the same list object. append, remove, pop, sort, and reverse modify the original list.

CHAPTER 9 - TUPLE IN PYTHON

Core idea

A **tuple** is an ordered collection like a list, but it is **immutable**.

```
values = (23, 45, 67, 43)
values2 = 23, 45, 67, 43     # parentheses are optional
```

Tuple operations

- Index and slice with square brackets: values[0], values[1:3].
- Use len(), min(), max(), and sum() where values support them.
- The methods highlighted are count() and index().
- Methods that change a list, such as append() or remove(), do not exist for tuples.
- Item assignment causes a TypeError because a tuple does not support item assignment.

Packing and unpacking

```
tup = (10, "Naven", 20)
a, name, b = tup
```

The number of target variables must match the number of tuple values. Otherwise Python reports an unpacking error (for example, not enough values to unpack).

Nested collections

A tuple can contain lists or other tuples. The tuple's own element references cannot be replaced, but a mutable list stored inside it can still be changed.

One-element tuple

```
single = (45,)
```

The comma is essential. `(45)` is just the integer `45` in parentheses, not a tuple.

When to use

Use a tuple when the grouped data should remain fixed / should not be modified as a collection.

CHAPTER 10 - SET IN PYTHON

Core idea

A **set** is a collection of unique values. Duplicate entries are removed. A set is unordered, so do not rely on its display order or insertion order.

```
numbers = {10, 20, 20, 30}
print(numbers)      # each value appears once
```

Creating sets

- Non-empty set: `{1, 2, 3}`.
- Empty set: `set()`.
- `{}` creates an empty dictionary, not an empty set.
- `set(iterable)` can create a set from another iterable.

Membership and length

```
21 in numbers      # True/False
len(numbers)
```

The transcript explains that sets use hash-based storage, which supports efficient membership checks.

Set mathematics

Let `A` and `B` be sets:

Operator	Name	Meaning
`A - B`	Difference	In A but not B.
`A | B`	Union	All distinct values from A and B.
`A & B`	Intersection	Values common to both.
`A ^ B`	Symmetric difference	Values in either set, but not in both.

The transcript demonstrates these with letters and vowels; repeated letters collapse into one set element.

CHAPTER 11 - DICTIONARY IN PYTHON

Core idea

A **dictionary** stores data as key-value pairs. Lists use numeric positions; dictionaries let us choose meaningful keys.

```
person = {"name": "Naven", "age": 30}
```

- Keys are unique.
- Keys may be strings or integers (as demonstrated in the lesson).
- Values can be numbers, strings, lists, or other dictionaries.
- Lookup uses the key, not a positional index.

Read and update

```
person["name"]          # lookup by key
person["age"] = 31      # update value
```

Accessing a missing key with `person["city"]` raises `KeyError`.

Safer lookup with `get()`

```
person.get("city")
person.get("city", "not found")
```

`get()` returns `None` when the key is absent unless a default is supplied.

Keys and values

The transcript demonstrates constructing a dictionary from a sequence of keys and matching values:

```
keys = ["a", "b", "c"]
values = [10, 20, 30]
data = dict(zip(keys, values))
```

`zip()` pairs items by position. If the source is an unordered set, pairing order may not match an intended order; use an ordered collection when order matters.

Delete and remove

```
removed = person.pop("age")
del person["name"]
```

- `pop(key)` removes the key and returns its value.
- `del` removes the specified dictionary entry.

Nested dictionaries

A value may itself be a dictionary. The transcript uses programming-language/IDE information and nested Java information as an example:

```
tools = {
    "Java": {"Core Java": "VS Code", "Spring": "IntelliJ IDEA"},
    "Python": "VS Code"
}
print(tools["Java"]["Spring"])
```

Nested lookup chains the keys: `outer[key][inner_key]`.

CHAPTER 12 - VARIABLE STORAGE IN PYTHON

Core idea

In Python, values are **objects**, and variable names refer to those objects. id() is used in the transcript to inspect object identity.

```
a = 5
b = 5
print(id(a), id(b))
```

The transcript demonstrates that small integer values can be cached/reused, so names assigned the same small integer may show the same identity.

Reassignment is not mutation

```
a = 5
b = 6
b = 9
print(a)     # 5 remains unchanged
print(b)     # 9
```

Changing b to 9 rebinds the name; it does not change the object/value referred to by a.

References and garbage collection

- Several variable names can refer to the same object.
- If a name is reassigned, the old object may still remain while another reference points to it.
- When an object has no references left, it can be cleaned up by **garbage collection**.
- The transcript compares this to cleaning a room: keep items that are still being used/referred to, discard unreferenced items.

String interning and integer caching

- The transcript demonstrates that repeated short strings (for example, the same short name) may refer to the same interned string object.
- Longer phrases may not be interned in the same way by default, so identical long strings can have different identities.
- Small integers may be cached; large values (the transcript demonstrates 1000) may be represented by different objects.
- These are implementation/storage behaviours; use equality (==) to compare values, not id().

CHAPTER 13 - DATA TYPES IN PYTHON

Check a value's type

```
x = 10
print(type(x))
```

The transcript treats types as classes and introduces type() to inspect them.

Numeric types

Type	Example	Notes from the lesson
int	10	Whole-number integer.
float	3.14	Decimal/point value.
complex	6 + 8j	Python uses j for the imaginary part, not i.

```
c = 6 + 8j
print(type(c))
a, b = 2, 3
c = complex(a, b)
```

Type conversion

```
pi = 3.14
k = int(pi)      # 3: fractional part is discarded
p = float(k)     # 3.0
```

The transcript emphasises that converting a float to int loses the fractional part; it truncates rather than rounding.

Boolean and None

- bool has two values: True and False.
- Relational expressions produce Boolean results.
- int(True) is 1; int(False) is 0.
- Python uses None for the absence of a value (similar to null terminology in other languages).

```
result = None
print(result)    # None
```

Collection / sequence types revisited

The transcript reviews:

- list
- tuple
- set
- dict
- range

range(10) represents values from 0 to 9; the end value is excluded. To include 10, use an end value of 11.

```
list(range(10))         # 0 through 9
list(range(2, 11, 2))   # 2, 4, 6, 8, 10
```

range(start, stop, step) uses a default start of 0 and step of 1 when omitted. The transcript uses step 2 to generate even numbers.

CHAPTER 14 - OPERATORS IN PYTHON

Arithmetic and assignment operators

Operator	Use
+, -, *, /	Addition, subtraction, multiplication, division.
//	Floor/quotient division.
%	Remainder/modulo.
**	Power.
=	Assignment.
+=	Add and assign (shorthand).

```
a = 3
b = 2
c = a + b
```

```
a = a + 2
a += 2      # shorthand form
```

Python does not use a++ as the increment syntax in this lesson; use a += 1 or a = a + 1.

Multiple assignment

```
x, y = 6, 5
```

The transcript demonstrates assigning two values to two variables in one statement.

Binary vs unary operators

- **Binary operator:** works with two operands, e.g. a + b.
- **Unary operator:** works with one operand, e.g. -a.
- -a produces the negative expression. To update the variable itself, assign it: a = -a.

Relational operators

<, >, <=, >=, ==, != compare values and return True or False.

Do not confuse:

- = assigns a value.
- == checks equality.
- != means not equal (! is used as part of this relational operator in Python).

Logical operators and truth table

- and: result is True only when **both** conditions are True.
- or: result is True when **at least one** condition is True; it is False only when both are False.
- not: reverses a Boolean result.

A	B	A and B	A or B
True	True	True	True
True	False	False	True
False	True	False	True
False	False	False	False

```
a = 4
b = 5
result = a < 10 and b > 1
print(result)
print(not result)
```

The transcript uses an umbrella/rain example to explain combining conditions: choose and or or according to whether all or only one of the conditions must be satisfied.

PART B - FUNCTIONS, INPUT, AND CONTROL FLOW

CHAPTER 15 - INTRODUCTION TO FUNCTION

Core idea

A function groups reusable statements under a name. It prevents repeated code and makes a project easier to organise.

Function structure

```
def add_numbers():
    a = 5
    b = 4
    c = a + b
    print(c)

add_numbers()
```

- def starts a function definition.
- The function name is followed by parentheses and a colon.
- The indented statements form the function body.
- A function runs when it is called; defining it alone does not execute its body.
- The # symbol begins a comment; Python ignores the comment during execution.

Parameters, arguments, and return

```
def add(a, b):
    return a + b

result = add(5, 4)
print(result)
```

- **Parameters** (a, b) are names in the function definition.
- **Arguments** (5, 4) are values supplied at the call.
- return sends a value back to the caller.
- print() displays a value; it is not the same as returning it.
- If a function has no explicit return, its result is None.

Docstring

A triple-quoted string immediately inside a function can describe what that function does. The transcript demonstrates that an IDE can show this documentation when hovering over the function.

```
def add(a, b):
    """Add two values and return the result."""
    return a + b
```

Running a project in VS Code

- Open one project folder.
- Create demo.py and test with print("Hello").
- Run with the play button or python demo.py in the terminal.
- The transcript also shows how pwd (present working directory) confirms which folder the terminal is in and mentions that a PowerShell prompt can be shortened with a prompt function/script.

CHAPTER 16 - WORKING WITH MODULE WITH math

Core idea

A **module** groups related code. Instead of writing every mathematical operation yourself, import functions from Python's math module.

```
import math
print(math.sqrt(25))       # 5.0
```

Import styles

```
import math
math.sqrt(25)

from math import sqrt
sqrt(25)
```

- import math keeps the module name as a prefix.
- from math import sqrt imports one selected function so it can be called directly.

Functions demonstrated

```
math.sqrt(25)      # square root
math.ceil(59.44)   # 60: round upward
math.floor(59.9)   # 59: round downward
```

The transcript introduces ceil() and floor() as examples of other functions available from the module.

CHAPTER 17 - SWAPPING OF VARIABLES IN PYTHON

The problem

Suppose a = 5 and b = 6. Simply doing a = b followed by b = a loses the original value of a; both become 6.

Method 1: temporary variable

```
a = 5
b = 6
c = a
a = b
b = c
```

The temporary variable c preserves the original value of a.

Method 2: arithmetic swap

```
a = a + b
b = a - b
a = a - b
```

This avoids a temporary variable but uses more arithmetic operations and can make values larger during the calculation.

Method 3: Python tuple unpacking

```
a, b = b, a
```

This is the concise Python style shown in the transcript. Python evaluates the right-hand values before assigning them to the left-hand variables.

Method 4: XOR / bitwise mention

The transcript briefly mentions a bitwise XOR-style swap as another approach, but the main practical focus is on the temporary-variable and tuple-unpacking methods.

CHAPTER 18 - USER INPUT

Core idea

input() pauses the program and collects text typed by the user.

```
name = input("Enter your name: ")
print(name)
```

The prompt text is important: without it, the user may not know what the application is waiting for.

input() returns a string

Even when a user types digits, input() initially returns text.

```
a = input("Enter first number: ")
b = input("Enter second number: ")
print(a + b)            # concatenates strings, e.g. "5" + "7" -> "57"
print(int(a) + int(b))   # numeric addition -> 12
```

Use int() for integer input or float() when decimal input is required.

Read only the first character

The transcript demonstrates that you can use indexing/slicing on the returned string:

```
choice = input("Enter a character: ")[0]
```

This keeps the first character from the user's input.

CHAPTER 19 - if STATEMENT

Core idea

An if statement runs an indented block only when its condition evaluates to True.

```
number = 8
if number % 2 == 0:
    print("Even")
```

Syntax reminders

- Write a condition after if.
- End the condition line with a colon :.
- Indentation defines the code block; the transcript recommends four spaces (or consistent indentation).
- Python uses indentation rather than braces to mark the block.
- Statements outside the indented block execute independently of the condition.

Even/odd check

```
if number % 2 == 0:
```

```
    print("Even")
if number % 2 == 1:
    print("Odd")
```

The remainder operator % is used to test divisibility by 2.

CHAPTER 20 - else STATEMENT AND DEBUGGING

if + else

Use else when there are two mutually exclusive paths: one for True and the other for False.

```
number = 7
if number % 2 == 0:
    print("Even")
else:
    print("Odd")
```

Only the matching branch runs.

Debugging: find the path and values

The transcript introduces debugging as a way to examine why a program produces unexpected output or fails.

- Set a **breakpoint** on a line.
- Start the debugger.
- Execution pauses at the breakpoint.
- Use step controls to move through the program one statement at a time.
- Observe variable values and which branch executes.
- Continue/stop execution after inspecting the issue.

Main lesson

A program can have valid syntax but still have the wrong logic. Debugging helps inspect the actual path and values rather than guessing.

CHAPTER 21 - NESTED if

Core idea

A nested if is an if statement inside another if/else block. The inner condition is checked only when execution enters the outer block.

Example pattern from the transcript:

```
if number % 2 == 0:
    if salary > 5:
        print("Great job")
    else:
        print("Try again")
else:
    print("Number is odd")
```

The inner decision depends on the outer decision. Indentation is especially important because it tells Python which if owns each else.

CHAPTER 22 - elif STATEMENT

Core idea

Use elif (else-if) when there are multiple possible conditions.

```
number = 3
if number == 1:
    print("One")
elif number == 2:
    print("Two")
elif number == 3:
    print("Three")
else:
    print("Other value")
```

if vs elif

- Multiple independent if statements are all checked separately.
- In an if/elif chain, Python stops after the first matching branch.
- There can be multiple elif statements, but only one initial if and one optional final else in a chain.
- else catches values not matched by the earlier conditions (for example, 6 or a negative number in the transcript's number-to-word exercise).

CHAPTER 23 - match STATEMENT

Core idea

The transcript introduces match as a switch/case-like construct available from Python 3.10.

```
number = 2
match number:
    case 1:
        print("One")
    case 2:
        print("Two")
    case 3:
        print("Three")
    case _:
        print("Other value")
```

- match evaluates the subject value.
- Each case represents a pattern/value to match.
- case _ is the wildcard/default branch.
- The transcript presents it as convenient for matching one of several values; if/elif remains useful for more general conditions.

CHAPTER 24 - while LOOP

Core idea

A while loop repeats its indented block while its condition remains True.

```
i = 1
while i <= 5:
    print(i)
    i += 1
```

The counter starts at 1; each iteration prints and increments. When i becomes 6, the condition is False and the loop stops.

Avoid an infinite loop

If the counter/condition never changes, the loop may run forever and repeatedly execute the same work. The transcript intentionally warns about the CPU impact of infinite loops.

Trace table

Iteration	i at condition	i <= 5	Printed	i after increment
1	1	True	1	2
2	2	True	2	3
3	3	True	3	4
4	4	True	4	5
5	5	True	5	6
Exit check	6	False	-	-

Nested while loops

The transcript demonstrates an outer loop that prints a word (such as TalisCoCo) five times and an inner loop that prints another word (such as rocks) four times for each outer cycle.

- The inner counter must be reset inside the outer loop so the inner loop can run again for each outer iteration.
- If it is initialized only once before the outer loop, it may remain beyond its limit after the first run, and later inner loops will not execute.
- print(..., end="") prevents the automatic newline; a separate print() can create a new line after the inner loop.

CHAPTER 25 - for LOOP

Why for?

The transcript compares while and for loops. A while loop needs a counter, a condition, and an increment. A for loop is shorter when working through a collection or range.

Iterate through a collection

```
data = [2, "Naven", 4.5, 8, "TalisCoCo", "Python"]
for value in data:
    print(value)
```

- value is the loop variable; its name can be changed.
- data is the iterable/collection.
- Python supplies one value at a time to value.

Equivalent while pattern

```
i = 0
while i < len(data):
    print(data[i])
    i += 1
```

The transcript first tries an explicit final index, then explains why it is safer to use len(data) and stop when i < len(data) (the final index is length minus one).

range() with for

```
for value in range(10):
    print(value)       # 0 through 9
```

range supplies the values one by one. It is useful when the number of iterations is known or when you want a generated sequence.

Choosing a loop

- **for:** convenient for a known iterable, list, collection, or range.
- **while:** useful when repetition depends on a condition and the number of iterations is not known in advance.

CHAPTER 26 - break AND continue

Core idea

Both statements change the normal flow of a loop.

- break ends the nearest enclosing loop immediately.
- continue skips the remaining statements in the current iteration and moves to the next iteration.

Skip selected values with continue

The transcript's example uses a range and skips values divisible by 3 (0, 3, 6, 9):

```
for value in range(10):
    if value % 3 == 0:
        continue
    print(value)
```

Stop at a selected value with break

```
for value in range(10):
    if value == 6:
        break
    print(value)
```

The loop ends as soon as the break condition is met. The remaining values are not processed.

Remember the distinction

continue = skip this turn.
break = leave the loop.

PART C - ARRAYS AND FUNCTIONAL PYTHON

CHAPTER 27 - ARRAY

Why introduce an array?

A list can hold different types, which is flexible, but the transcript raises two concerns for numeric processing:

85. A mixed-type list makes it harder to apply one uniform operation to every value.
86. List values may not be stored as one compact block in memory, which can make some operations/search work less efficient.

An **array** is a collection intended for values of one declared type. Python's built-in array module supports typed arrays. For more complex or multidimensional numerical work, the transcript mentions **NumPy**.

Import and type codes

```
from array import array
```

The first argument to the array constructor is a type code; the values must match that type.

Type code mentioned	Meaning described in the transcript
b	Signed byte.
B	Unsigned byte.
h	Signed short.
i	Signed integer.
I	Unsigned integer.
l / L	Long integer variants.
f	Float.
d	Double/float variant.

Create and display an array

```
from array import array
arr1 = array('i', [10, 20, 30, 40])
print(arr1)
for value in arr1:
    print(value)
```

The type code is compulsory. The transcript demonstrates that a value of the wrong type (for example a float in an integer array) causes a type error.

Buffer information

```
print(arr1.buffer_info())
```

The transcript identifies the returned information as the array's memory address and size/length information.

CHAPTER 28 - ARRAY FUNCTIONS

Methods demonstrated

```
arr1.append(77)        # add at end
arr1.reverse()         # reverse in place
arr1.count(20)         # count occurrences
arr1.index(30)         # find an index
arr1.insert(1, 15)     # insert at a position
arr1.pop()             # remove and return last item
arr1.remove(20)        # remove by value
arr1.tolist()          # return a list of values
```

The transcript explores methods through IDE autocomplete and the array module's available members.

Reference vs independent copy

```
arr2 = arr1
```

This does **not** copy the array. arr1 and arr2 refer to the same object; changing an element through one name is visible through the other.

To create a separate array from the current values:

```
arr2 = array(arr1.typecode, arr1.tolist())
```

The transcript then improves this with a generator expression so that an intermediate list is not created:

```
arr2 = array(arr1.typecode, (n for n in arr1))
```

This copies values one by one and is presented as more memory-efficient, while requiring iteration work.

Practice task from the transcript

- Create an empty typed array.
- Ask the user how many values they want to enter.
- Use a loop to collect that many values.
- Append each input value to the array.

CHAPTER 29 - ARGUMENTS IN FUNCTION

Positional arguments

```
def add(num1, num2):
    return num1 + num2

result = add(4, 5)
print(result)       # 9
```

The position of each argument determines which parameter receives it. If a required positional argument is missing, Python raises a TypeError.

Default arguments

```
def add(num1=0, num2=0):
    return num1 + num2
```

If the caller omits a value, the default is used. The transcript tests:

- No arguments -> 0.
- One argument such as 4 -> 4.
- Two arguments such as 4, 5 -> 9.
- More arguments than the function accepts -> error.

When a parameter has a default value, parameters after it also need defaults in the normal positional-parameter arrangement shown in the lesson.

Why defining many versions is not the answer

The transcript considers creating multiple add() definitions for different numbers of parameters, but that does not scale. Repeatedly defining the same function name replaces the earlier definition rather than behaving like method overloading in some other languages.

Variable-length positional arguments: *args

```
def add(num1, *num2):
    total = num1
    for n in num2:
        total += n
    return total

print(add(4, 9, 6, 7))
```

- The first value goes into num1.
- The remaining values are collected into a tuple named num2.
- Loop over that tuple and accumulate the total.
- The method is: one fixed first argument + a tuple containing all remaining arguments. The exact sample values in the recording are less important than understanding how *num2 collects the extra values.

Person example and keyword arguments

```
def person(name, age=18):
    print("Name:", name)
    print("Age:", age)

person("Naven", 30)
person("Naven")
person(age=30, name="Naven")
```

- age=18 acts as a default when age is omitted (the transcript uses an adult-rated movie example).
- Positional order matters when passing ordinary positional arguments.
- Keyword arguments identify the destination parameter by name; they make the call explicit and allow a different order.

CHAPTER 30 - GLOBAL VS LOCAL VARIABLES

Local variable

A variable created inside a function is local to that function unless explicitly declared otherwise.

```
x = 20

def show():
    x = 15
    print("Inside:", x)

show()
print("Outside:", x)
```

The function prints 15, while the global value remains 20.

Global variable

Use the global keyword when a function needs to assign to the module-level variable:

```
x = 20

def update():
    global x
    x = 15
    print("Inside:", x)

update()
print("Outside:", x)
```

Now the outer/global value changes too. The transcript contrasts local and global scope and shows why declaring global x is necessary when assigning to the global name inside the function.

CHAPTER 31 - FACTORIAL IN PYTHON

Definition

For a positive integer n, factorial is the product of all positive integers from 1 through n.

```
5! = 5 x 4 x 3 x 2 x 1 = 120
```

The same multiplication can be written in ascending order: 1 x 2 x 3 x 4 x 5.

Loop approach

```
def factorial(n):
```

```
    result = 1
    for value in range(1, n + 1):
        result *= value
    return result

print(factorial(5))      # 120
```

- Start the accumulator at 1.
- Run from 1 through `n` (the range stop is `n + 1` because it is excluded).
- Multiply the accumulator by each value.

The transcript uses this exercise to practise loop logic and shows that Python can calculate large factorial values (including much larger numbers than 20 or 30).

CHAPTER 32 - RECURSION

Definition

Recursion is a function calling itself.

```
def greet():
    print("Hello")
    greet()

greet()
```

A function definition is not executed until called. When `greet()` calls itself, it prints repeatedly and keeps adding calls.

Recursion limit

Uncontrolled recursion eventually raises `RecursionError` because Python limits the recursion depth. The transcript demonstrates checking/changing the setting using `sys`:

```
import sys
print(sys.getrecursionlimit())
sys.setrecursionlimit(3000)
```

The recording observes a default value of 1000 in its environment. It also demonstrates higher values, but increasing the limit does not make uncontrolled recursion safe; each call consumes resources.

Slow the demonstration

```
from time import sleep
sleep(0.02)
```

A short sleep can make repeated output easier to observe. The transcript also shows a counter and uses `global count` when the recursive function must update a global counter.

Main warning

Recursion must have a stopping condition. Otherwise the function keeps calling itself until it reaches the recursion limit or causes resource problems.

CHAPTER 33 - FACTORIAL USING RECURSION

Recursive relationship

```
5! = 5 x 4!
4! = 4 x 3!
```

```
3! = 3 x 2!
2! = 2 x 1!
1! = 1
```

Each call reduces the problem until the base condition is reached. Then the answers return through the waiting calls.

Code

```
def fact(num):
    if num == 1:
        return 1              # base case
    return num * fact(num - 1)   # recursive call

print(fact(5))                # 120
```

Call-stack walkthrough for **fact(5)**

87. fact(5) waits for fact(4).
88. fact(4) waits for fact(3).
89. fact(3) waits for fact(2).
90. fact(2) waits for fact(1).
91. fact(1) returns 1.
92. Return/unwind: 2 x 1 = 2, 3 x 2 = 6, 4 x 6 = 24, 5 x 24 = 120.

Debugger observation

The transcript uses **Step Into** and the call stack to show several active calls, each with its own local num value. As each call returns, one stack frame disappears and the previous call continues.

PART D - FUNCTIONAL PROGRAMMING, MODULES, AND DECORATORS

CHAPTER 34 - HIGHER-ORDER FUNCTION

Core idea

A **higher-order function** accepts another function as an argument (or returns a function). Python lets functions be passed around like values.

First example: square and cube

```
def square(num):
    return num * num

def cube(num):
    return num * num * num

result = square(5)
print(result)       # 25
```

A normal function call directly names the operation. The transcript then makes the operation configurable.

Pass a function into another function

```
def operate(num, operation):
```

```
    return operation(num)

print(operate(5, square))
print(operate(5, cube))
```

- square and cube are passed without parentheses, because the function itself is being passed rather than called immediately.
- Inside operate, operation refers to whichever function was supplied, so operation(num) calls it.
- operate is higher-order because it accepts another function.

The transcript expands the example to a list of numbers ([5, 6, 7]), loops over the values, applies the selected function to each value, and prints each result.

Why Python supports this

The transcript describes Python as supporting functional programming and calls functions **first-class citizens**: functions can be created, called, assigned, passed as arguments, and returned. It also notes that functions are objects internally.

CHAPTER 35 - ANONYMOUS FUNCTION USING lambda

Core idea

A **lambda** is a short anonymous function, useful when a small function is needed in one place.

```
square = lambda n: n * n
cube = lambda n: n * n * n
print(square(5))
```

Syntax pattern

```
lambda parameters: expression
```

- No def statement or function name is required for the inline expression.
- The expression's value is returned automatically.
- The transcript replaces a one-line square/cube function with a lambda and passes the lambda into a higher-order function.

When it is useful

Use lambda for short expressions passed to another function (such as filter, map, or reduce). Use a normal def when the logic needs several statements or a descriptive reusable function body.

CHAPTER 36 - filter() FUNCTION

Core idea

filter(function, iterable) selects the values for which the supplied function returns True.

```
def is_even(n):
    return n % 2 == 0

nums = [1, 2, 3, 4, 5, 6, 7, 8]
evens = list(filter(is_even, nums))
print(evens)       # [2, 4, 6, 8]
```

What happens internally

- filter() takes a function and an iterable.

- It supplies one element at a time to the function.
- A True result keeps the value; a False result excludes it.
- In the transcript, filter() itself is shown as a filter object; wrap it in list() to display/collect the selected values as a list.

Lambda version

```
evens = list(filter(lambda n: n % 2 == 0, nums))
```

The transcript explains that a lambda is convenient because the even-check function is a small one-line condition used only for filtering.

CHAPTER 37 - map() AND reduce() FUNCTIONS

map() transforms every value

The transcript first creates a filtered list of even numbers and then wants to double every selected value.

```
def double_it(n):
    return n * 2

doubles = list(map(double_it, evens))
```

- map(function, iterable) applies the function to each item.
- The output contains one transformed value for each input value.
- As with filter, the transcript converts the map object to a list for display.

Lambda version:

```
doubles = list(map(lambda n: n * 2, evens))
```

reduce() combines values into one result

The transcript sums the doubled values. reduce is imported from functools:

```
from functools import reduce

def sum_it(a, b):
    return a + b

total = reduce(sum_it, doubles)
print(total)
```

- reduce() applies a two-argument function cumulatively.
- It combines the first two values, then combines that result with the next value, and continues until one value remains.

Lambda version:

```
total = reduce(lambda a, b: a + b, doubles)
```

Transcript's data flow

nums -> filter(even values) -> map(double each even value) -> reduce(sum all doubled values)

Example with even numbers [4, 2, 6, 8]:

- Doubles: [8, 4, 12, 16]
- Reduced sum: 40

CHAPTER 38 - INNER FUNCTION

Core idea

An **inner function** is defined inside another function.

```
def outer():
    print("In outer function")

    def inner():
        print("In inner function")

    inner()

outer()
```

- Calling `outer()` runs the outer statements.
- `inner` is defined inside `outer` and is available from that scope.
- Calling `inner()` directly from outside does not work because the name is local to the outer function.

Return a function

The transcript then demonstrates returning the inner function itself:

```
def outer():
    def inner(value):
        print("In inner function", value)
    return inner

something = outer()
something(5)
```

Important distinction:

- `return inner` returns the function object.
- `return inner()` calls the function immediately and returns its result (which may be `None` if there is no explicit return).
- `something` receives the returned function and can then be called.

Why this matters

Inner functions can keep a helper restricted to an outer operation. The transcript introduces them here because they are used to build decorators.

CHAPTER 39 - DECORATORS IN PYTHON

Core idea

A **decorator** takes a function, wraps it with additional behaviour, and returns a replacement function. The original function can then be called with the new behaviour without rewriting its body.

Start with an ordinary function

```
def divide(a, b):
    return a / b
```

The transcript's first problem is to ensure the larger value is the first value before dividing (a “greater first” behaviour).

Decorator pattern

```
def greater_first(func):
```

```
    def wrap(a, b):
        if a < b:
            a, b = b, a
        return func(a, b)
    return wrap
```

- func is the original function being decorated.
- wrap is the inner function that changes/controls the arguments.
- The wrapper calls the original function with the resulting arguments.
- return wrap returns the wrapper function, not the result of calling it.

Apply a decorator

```
@greater_first
def divide(a, b):
    return a / b
```

The transcript also shows the equivalent assignment style:

```
def divide(a, b):
    return a / b

divide = greater_first(divide)
```

The @decorator syntax is the modern/clean way demonstrated.

Logging decorator

The transcript adds another decorator to log:

- The arguments received.
- The result of calling the function.
- The function's result is returned to the caller.

```
def log_deco(func):
    def wrap(a, b):
        print("Values received:", a, b)
        result = func(a, b)
        print("Result:", result)
        return result
    return wrap
```

Apply @log_deco above the functions that need logging. If it is applied only to sub, it logs only sub; apply it to divide or add as needed.

Generic arguments

A wrapper fixed to (a, b) will fail when the decorated function accepts three or more arguments. The transcript returns to the variable-length argument idea and explains using *args and keyword arguments (**kwargs) to make wrappers more generic:

```
def decorator(func):
    def wrap(*args, **kwargs):
        # additional behaviour
        return func(*args, **kwargs)
    return wrap
```

This pattern allows ordinary positional and keyword arguments to pass through the wrapper.

CHAPTER 40 - MODULES AND PACKAGES

Module

A **module** is a Python file that contains related code (functions, variables, classes) which can be reused by another file.

Package

A **package** is a folder used to group related Python modules. The transcript creates a `calculator` package and puts files such as `calc.py` and `demo.py` inside it.

Example structure

```
project/
   calculator/
      calc.py
      demo.py
```

In `calc.py`:

```
def add(a, b):
   return a + b

def sub(a, b):
   return a - b
```

In `demo.py`, import the module before using its members:

```
import calc
print(calc.add(5, 4))
```

Import styles

```
import calc
calc.add(5, 4)

from calc import add, sub
add(5, 4)
sub(5, 4)
```

- `import calc` keeps the module prefix.
- `from calc import add` imports only the named function(s), so the module prefix is not needed for those names.
- `from calc import *` imports all names (shown in the transcript as a wildcard option).

Package path matters

- A file inside the same package can import a sibling module using its module name.
- A file outside the package needs to include the package name in the import path, e.g. `from calculator import calc` (or `from calculator.calc import add`).
- The terminal's working directory matters. The transcript changes into the package folder to run its internal `demo.py`, then returns to the project root to run an outside file.

CHAPTER 41 - SPECIAL VARIABLE `__name__`

Core idea

The transcript introduces the special variable `__name__` and its common use for controlling whether code runs when a file is executed directly or imported.

```
if __name__ == "__main__":
    # code to run only when this file is the entry point
    print("Run directly")
```

- When Python runs a file directly, its __name__ is set to "__main__".
- When the file is imported as a module by another file, __name__ is the module name.
- This lets a module define reusable functions/classes without automatically running its demo/test code on import.
- The transcript uses this check before creating objects and executing a demonstration in the multithreading chapter.

PART E - OBJECT-ORIENTED PROGRAMMING

CHAPTER 42 - OOP INTRODUCTION

Functional vs object-oriented thinking

- In functional programming, functions are the first thing we organise around; functions can be passed, returned, and reused.
- In object-oriented programming (OOP), the **object** is the central idea.
- Python supports both styles; real applications may use OOP to represent things and functions for processing.

What is an object?

The transcript uses everyday examples: a phone, AC remote, microphone, camera, and employee.
An object has:

93. **Properties / attributes:** what it knows or has.
94. **Behaviour / methods:** what it does.

Employee example:

- Properties: name, designation, salary, location, home location.
- Behaviours: work, communicate, and other actions.

Remote example:

- Properties: colour, size, buttons.
- Behaviour: control lights.

Class is the blueprint

A factory needs a blueprint before making a remote. Similarly, a **class** defines what an object will contain and do.

- A class is a blueprint.
- An object is an instance created from that class.
- One class can produce multiple objects.
- Variables in a class represent attributes/properties.
- Functions in a class are called **methods**.

CHAPTER 43 - CLASS AND OBJECT

Basic syntax

```
class Computer:
    def config(self):
        print("i5, 16GB, 1TB")

com1 = Computer()
```

```
com2 = Computer()
com1.config()
com2.config()
```

- class defines a class.
- Computer() creates an object (instance).
- com1 and com2 are different object references.
- A method is a function defined inside a class.

self

- self refers to the current object.
- When calling com1.config(), Python supplies com1 as the first argument to the method.
- The transcript also shows the equivalent unbound-style call Computer.config(com1) to make the object argument visible.

Attributes and method behaviour

The transcript uses a computer as an object and discusses storing details such as processor, RAM, and storage, while methods perform actions. Values can be accessed through the object's attribute names.

Project organisation

The transcript recommends placing related files in packages rather than leaving every file in one large project folder (for example, keeping calculator work and basic/OOP examples in separate packages).

CHAPTER 44 - __init__() METHOD

Core idea

__init__() is the initialisation method called automatically after an object is created. It is used to assign starting values to instance attributes.

```
class Computer:
    def __init__(self, cpu, ram):
        self.cpu = cpu
        self.ram = ram

    def config(self):
        print("Configuration:", self.cpu, self.ram)

com1 = Computer("i5", 16)
com2 = Computer("Ryzen", 8)
com1.config()
com2.config()
```

- self.cpu and self.ram belong to the object.
- cpu and ram are values supplied when creating the object.
- Each object can have different attribute values.
- A local variable belongs to the function call in which it is created. It is separate from a global variable with the same name.

CHAPTER 45 - CONSTRUCTOR USING __new__() METHOD

Core idea

The transcript introduces __new__() as the method that actually creates/returns a new instance, while __init__() initialises that created instance.

Creation and initialisation sequence

95. `__new__(cls, ...)` is called to create an object.
96. It returns the instance.
97. `__init__(self, ...)` receives that instance and initialises its attributes.

Pattern demonstrated

```
class Demo:
    def __new__(cls, *args, **kwargs):
        print("Creating object")
        return super().__new__(cls)

    def __init__(self, value):
        print("Initialising object")
        self.value = value

obj = Demo(10)
```

The main point is the distinction: `__new__` is related to object creation; `__init__` is related to setting up the object's state after creation. The transcript also deliberately removes the `return` from an overridden `__new__()` to show what goes wrong: if `__new__()` does not return a valid instance, the result is not a usable `Demo` object (the demonstrated failure is an attribute access on `NoneType`), and normal initialisation does not proceed. In ordinary classes, use `__init__()` for setting attributes; override `__new__()` only when you need to control object creation itself.

CHAPTER 46 - TYPES OF METHODS

1. Instance method

- Works with an object/instance.
- Its first parameter is conventionally `self`.
- It can access and update instance attributes.

```
class Student:
    def show(self):
        print("Instance method")
```

The transcript's `Computer` example uses `cpu`, `ram`, and `ssd` as instance attributes created with `self`. Each computer object can have different values. It also declares `brand` at class level; this is a **class variable**, shared by objects unless an instance creates its own attribute with the same name.

2. Class method

- Works with class-level information rather than one particular object's data.
- Uses `@classmethod` and `cls` as its first parameter.
- The transcript shows that calling a method that expects `cls` without the `@classmethod` decorator causes a missing-argument problem.

```
class Computer:
    brand = "Dell"

    @classmethod
    def info(cls):
        return cls.brand

print(Computer.info())
```

3. Static method

- Does not receive `self` or `cls` automatically.
- Uses `@staticmethod`.

- It is grouped inside the class because it is related to that class, even though it does not depend on instance/class state.
- The transcript's example converts a gigabyte value into bytes.

```
class Computer:
    @staticmethod
    def gb_to_bytes(gb):
        return gb * 1024 * 1024 * 1024

print(Computer.gb_to_bytes(4))
```

Quick comparison

Method type	Decorator	First parameter	Main relationship
Instance	None	self	One object
Class	@classmethod	cls	Class itself
Static	@staticmethod	None automatically	Utility related to the class

CHAPTER 47 - INHERITANCE AND MRO

Inheritance

Inheritance allows one class to reuse properties and methods from another class.

- **Parent / base / super class:** class being inherited from.
- **Child / derived / subclass:** class that inherits.

Types demonstrated

98. **Single-level:** one child inherits from one parent.
99. **Multilevel:** a chain such as A -> B -> C.
100. **Multiple inheritance:** one child inherits from two parents, such as `class C(A, B)`.

```
class A:
    def show(self):
        print("A show")

class B:
    def show(self):
        print("B show")

class C(A, B):
    pass

obj = C()
obj.show()
```

MRO - Method Resolution Order

When an object requests a method, Python searches according to the Method Resolution Order:

101. Search the object's own class.
102. If not found, search the first parent in the inheritance order.
103. Continue through the remaining parents according to the class's MRO.

In the transcript's `C(A, B)` example, if both A and B define `show()`, A wins because A is listed first. Changing the order to `C(B, A)` changes the selected parent method. If B has no `show()` but A has one, Python finds the method in A.

CHAPTER 48 - __init__() AND super() WITH INHERITANCE

Every class inherits from object

The transcript explains that Python classes inherit from the built-in object class by default. This is why an instance has built-in methods beyond the ones written by the developer.

Constructor lookup in inheritance

- When creating a child object, Python first checks whether the child class defines __init__().
- If the child has its own __init__(), that is the initialiser called.
- If it does not define one, the parent initialiser is used.
- Only one __init__() is automatically selected in this simple inheritance path; to deliberately call both parent and child initialisation logic, use super().

Example

```
class A:
    def __init__(self):
        print("A init")

    def f1(self):
        print("f1 works")

class B(A):
    def __init__(self):
        super().__init__()
        print("B init")

    def f2(self):
        self.f1()
        print("f2 works")

obj = B()
obj.f2()
```

super()

- super() gives access to the next class in the method resolution order (the transcript explains it as representing the super/parent class in this example).
- super().__init__() runs the parent initialiser.
- Parent methods can also be accessed through super(), while self.parent_method() is another way to call an inherited method when appropriate.
- The transcript uses both self and super() to show calling f1() from f2().

CHAPTER 49 - POLYMORPHISM AND DUCK TYPING

Polymorphism

“Poly” means many and “morphism” means forms: one interface/operation can behave in different ways depending on the object or implementation.

Duck typing

The transcript's phrase: **“If it quacks like a duck, treat it as a duck.”** In programming, what matters is whether an object provides the method/behaviour being used, not simply whether it belongs to one exact class.

Laptop / desktop example

```
class Laptop:
    def build(self):
        print("Laptop builds")
```

```
class Desktop:
    def build(self):
        print("Desktop builds")

class Alien:
    def code(self, machine):
        print("Alien building")
        machine.build()

naven = Alien()
naven.code(Laptop())
naven.code(Desktop())
```

Both Laptop and Desktop work because each supplies the required build() method. The transcript then creates a Tablet with open_pdf() instead of build(); passing it to code() fails because the required behaviour is missing.

Main lesson

The caller needs a usable object with the expected behaviour. Laptop and Desktop both "quack" by providing build(); Tablet does not. This is duck typing and a form of polymorphic behaviour.

CHAPTER 50 - OPERATOR OVERLOADING

Core idea

The same operator can behave differently depending on the types/objects on which it operates.

```
5 + 6            # 11: numeric addition
"5" + "6"         # "56": string concatenation
```

The operator is the same (+), but its behaviour changes with the operands. The transcript connects this to polymorphism.

Magic methods

Python classes implement operator behaviour using special/double-underscore methods. For addition, the transcript introduces __add__():

```
class int:
    # conceptual built-in behaviour
    def __add__(self, value):
        ...
```

You normally write a + b; Python calls the underlying special method. The transcript calls the readable operator syntax "syntactic sugar" and warns not to replace normal operators with magic-method calls in everyday code just to imitate the implementation.

Custom Account example and __str__()

The transcript creates an Account class with name and balance, then creates two account objects. Printing an ordinary custom object initially displays its class and memory location. The transcript explains that print() uses string conversion and introduces __str__() to display useful account information instead.

```
class Account:
    def __init__(self, name, balance):
        self.name = name
        self.balance = balance

    def __str__(self):
        return f"{self.name}: {self.balance}"
```

Custom addition: __add__()

The transcript defines adding two accounts as creating a combined account with the total balance.

```
class Account:
    def __init__(self, name, balance):
        self.name = name
        self.balance = balance

    def __add__(self, other):
        total = self.balance + other.balance
        return Account("Combined account", total)
```

Comparing objects: __gt__()

The transcript then compares two account balances and uses the greater-than behaviour to decide which account has more money. `__gt__()` defines the meaning of `>` for these objects. Other comparison special methods mentioned include `__lt__()` and `__eq__()`.

```
    def __gt__(self, other):
        return self.balance > other.balance
```

Remember: `a + b` can invoke `a.__add__(b)`; `a > b` can invoke `a.__gt__(b)`. These special methods let a class define operator behaviour for its own objects.

CHAPTER 51 - METHOD OVERRIDING

Core idea

Method overriding occurs when a child class defines a method with the same name (and compatible call shape) as a method in its parent. The child implementation takes precedence for child objects.

```
class A:
    def show(self):
        print("In A show")

class B(A):
    pass

obj = B()
obj.show()          # inherited A.show()
```

If B defines its own `show()`, the child method overrides the parent's behaviour:

```
class B(A):
    def show(self):
        print("In B show")
```

Now `B().show()` prints the B version.

Transcript analogy

The instructor tells a phone story: first claiming the family phone as one's own, then getting a personal phone. The parent method is like the inherited phone; once the child has its own implementation, the child's version is used.

Link to __str__

The previous operator-overloading chapter introduces `__str__` from the base `object` behaviour. Defining `__str__` in a custom class overrides the default representation used by `print()`.

CHAPTER 52 - ABSTRACTION IN PYTHON

Definition

Abstraction means hiding complex implementation details from the user and exposing only the essential interface.

Car example

A driver uses the steering wheel, accelerator, brake, and gear controls. The driver does not need to know all internal engine details to drive the car. A different car may have a different engine, but the visible driving interface remains familiar.

Interface vs implementation

- **Interface:** declares what an object must provide; it describes required operations.
- **Implementation:** contains the actual code that performs those operations.
- A user works with the interface; the internal implementation can be changed behind it.

Abstract classes and abstract methods

The transcript implements abstraction with abstract classes and abstract methods (using the `abc` module). An abstract class describes a contract and is not meant to be instantiated directly.

Payment gateway example

An e-commerce checkout should call a standard payment operation, but the actual provider might change (the transcript gives Razorpay/Stripe/PayPal-like examples and a fictional provider).

The problem with tight coupling:

- If checkout is written for one provider's exact method name, switching providers may require changing checkout code.
- Providers might use inconsistent method names, so the application needs a shared standard.

Abstract contract pattern

```
from abc import ABC, abstractmethod

class PaymentGateway(ABC):
    @abstractmethod
    def pay(self):
        pass

class RazorPay(PaymentGateway):
    def pay(self):
        print("Paying using RazorPay")

class StorePay(PaymentGateway):
    def pay(self):
        print("Paying using StorePay")
```

- `PaymentGateway` is the abstract base/interface.
- `pay()` is an abstract method and must be implemented by concrete child classes.
- A concrete provider implements the standard method.
- A class that has not implemented the abstract method cannot be instantiated; the transcript demonstrates this error when the required method is missing.

Why abstraction is useful

It reduces dependence on one implementation and makes provider substitution easier: checkout can depend on the common payment interface rather than the details of one provider.

PART F - ERROR HANDLING AND CONCURRENCY

CHAPTER 53 - EXCEPTIONS IN PYTHON

Why handle exceptions?

Applications should not crash when unexpected input or environmental conditions occur. The transcript distinguishes a simple calculator from mission-critical software (aircraft, cars, MRI machines, and banking systems) where a crash can have serious consequences.

Three error categories

Error type	Meaning	Transcript example
Syntax / compile-time-style error	Code does not follow Python's syntax/grammar.	Misspelling def, invalid keyword, malformed statement.
Logical error	Code runs but produces the wrong answer.	Factorial code returns 240 instead of 120 because the base result/logic was wrong.
Runtime error / exception	Something goes wrong while the program is running.	Dividing by zero; a required file has been deleted by the user.

The transcript stresses that logical errors can be difficult to notice: output may look reasonable but be wrong. Testing, QA, production checks, and feedback are part of the process.

First division example

```
a = 5
b = 2
result = a / b
print(result)
print("End of program")
```

When `b = 0`, Python raises `ZeroDivisionError`; statements after the failure do not run. The transcript uses this to show why the application should handle expected runtime failures.

try and except

```
try:
    result = a / b
    print(result)
except ZeroDivisionError:
    print("Division by zero is not allowed")

print("End of program")
```

- Put the statement that may fail inside `try`.
- Add an `except` block to handle a matching exception.
- The transcript warns that a `try` must have at least one `except` or `finally` block.

Exception hierarchy and specific handlers

The transcript opens Python's exception hierarchy and introduces the common `Exception` base class plus specific exceptions such as `ZeroDivisionError` and `ValueError`.

A specific example with user input:

```
try:
    a = int(input("Enter numerator: "))
    b = int(input("Enter denominator: "))
    result = a / b
    print("Result:", result)
```

```
except ZeroDivisionError:
    print("Division by zero is not allowed")
except ValueError:
    print("Invalid input - enter numeric values only")
except Exception as e:
    print("Unexpected error:", e)

print("End of execution")
```

- ZeroDivisionError handles a zero denominator.
- ValueError handles an invalid conversion such as typing t when int() expects digits.
- Exception as e captures the exception object so its message can be printed.
- Put specific handlers before the general handler; Python executes the first matching except block and does not execute all of them.

finally

The transcript explains that files, database connections, and network connections are resources that should be closed even when an error occurs.

```
try:
    print("Resource opened successfully")
    # use the resource
except Exception as e:
    print("Error:", e)
finally:
    print("Resource closed")
```

- finally runs whether or not an exception occurs (including when an unhandled exception is propagating).
- It is a suitable place for cleanup such as closing a resource.
- A try statement can have except, finally, or both; it needs at least one of them.

CHAPTER 54 - MULTITHREADING IN PYTHON

What is a thread?

A thread is a path of execution within a process. The transcript begins with two simple tasks, printing hello and hi, to show the difference between sequential execution and concurrent execution.

Sequential execution first

The initial program calls one method and lets it finish, then calls the second method:

```
class Hello:
    def do(self):
        for i in range(5):
            print("Hello", i + 1)

class Hi:
    def do(self):
        for i in range(5):
            print("Hi", i + 1)

if __name__ == "__main__":
    t1 = Hello()
    t2 = Hi()
    t1.do()
    t2.do()
```

Output is grouped: five Hello lines, then five Hi lines. This is sequential because the first call completes before the next begins.

Why threads are useful

The transcript compares this with modern applications:

- A phone can keep a call running while WhatsApp or a cricket-score application is used.
- A game can move the player, move enemies, update the score, and play music while the game continues.
- These tasks should make progress concurrently instead of waiting for one entire activity to finish.

Two ways to define a thread task

The transcript says threads can be created using classes or using functions; it shows the class approach first and then the simpler function-target approach.

Thread example (class style)

The transcript first changes the two task classes so they inherit from `threading.Thread`. The work is placed in a method named `run()`; calling `start()` starts a new thread and causes that thread to execute `run()`.

```
from threading import Thread

class Hello(Thread):
    def run(self):
        for i in range(5):
            print("Hello", i + 1)

class Hi(Thread):
    def run(self):
        for i in range(5):
            print("Hi", i + 1)

if __name__ == "__main__":
    t1 = Hello()
    t2 = Hi()
    t1.start()
    t2.start()
    t1.join()
    t2.join()
```

The transcript also shows the target-based style, where a function or bound method is supplied to `Thread(target=...)`.

Thread example (function style)

```
from threading import Thread
from time import sleep

def hello():
    for i in range(5):
        print("Hello", i + 1)
        sleep(0.5)

def hi():
    for i in range(5):
        print("Hi", i + 1)
        sleep(0.5)

if __name__ == "__main__":
    t1 = Thread(target=hello)
    t2 = Thread(target=hi)
    t1.start()
    t2.start()
    t1.join()
    t2.join()
```

- `Thread(target=...)` specifies which function the thread should execute.
- `start()` starts the thread.
- `join()` makes the main flow wait until that thread finishes.
- Output can interleave; the exact line order is not guaranteed.

Sleep and interleaving

The transcript adds a delay (sleep) to make the interleaving visible. Without a delay, a short function may complete too quickly to make the behaviour obvious. The transcript also uses the example of downloading multiple files.

I/O-bound work

For I/O-bound work such as downloading several files, one thread can wait for a network operation while another makes progress. The transcript shows three downloads (for example, file, image, and data) running in parallel; the demo reports a shorter total time than doing them serially. It also suggests experimenting with file sizes to make the duration dynamic.

CPU-bound work and the GIL

The transcript then changes the example to a CPU-heavy calculation: calculate the square of each number in a range and sum the squares. It compares one serial run with two threads, each assigned half the range.

The two-thread version does not become twice as fast. The explanation given is the **GIL (Global Interpreter Lock)**:

- In the described Python implementation, only one thread executes Python bytecode under the interpreter lock at a time.
- Threads can switch, but for CPU-bound Python work they do not necessarily use multiple CPU cores for simultaneous bytecode execution.
- Thread switching itself has some overhead.

Multiprocessing comparison

The transcript proposes multiprocessing for CPU-heavy work:

- A process has its own memory space and its own interpreter/GIL.
- Creating a process is heavier and uses more resources than creating a thread.
- Separate processes can run CPU work on separate cores (subject to the system/environment).

Pattern introduced in the transcript:

```
from multiprocessing import Process

def calculate(start, stop):
    total = 0
    for n in range(start, stop):
        total += n * n
    return total

if __name__ == "__main__":
    mid = 50_000_000 // 2
    p1 = Process(target=calculate, args=(0, mid))
    p2 = Process(target=calculate, args=(mid, 50_000_000))
    p1.start()
    p2.start()
    p1.join()
    p2.join()
```

The lesson's demonstration changes thread creation to process creation and compares timing; the recorded run reports the process approach completing faster than the serial/threaded CPU-bound variant, while also noting the extra memory and process-creation overhead.

Threads vs processes - final comparison

Situation	Transcript's guidance
I/O-bound work (downloads, waiting on network/file operations)	Threads are useful; they are lightweight and can overlap waiting time.
CPU-bound Python calculations	Threads may not speed up the work because of the GIL; consider multiprocessing or serial execution.
Need isolated memory / separate interpreter lock	Processes provide separate memory and their own GIL, but are

Situation	Transcript's guidance
	heavier to create and use more resources.

Final takeaway

Choose based on the kind of work, not just the word “parallel”:

- I/O waiting -> threads are often helpful.
- CPU-intensive Python calculation -> consider processes, while accounting for process overhead.

QUICK REVISION SHEETS

1. Python syntax at a glance

Need	Syntax / example
Display output	`print("Hello")`
Comment	`# comment`
Assign	`x = 10`
Read user input	`name = input("Name: ")`
Convert to integer	`n = int(input("Number: "))`
Define a function	`def fun(a, b):`
Return a result	`return a + b`
Define a class	`class Student:`
Initialise an object	`def __init__(self, name):`
Create an object	`obj = Student("Naven")`
Condition	`if x > 0:`
Else-if	`elif x == 0:`
Else	`else:`
Match	`match x:` / `case 1:`
While loop	`while condition:`
For loop	`for item in items:`
Import module	`import math`
Import selected member	`from math import sqrt`
Handle exception	`try:` / `except ValueError:` / `finally:`
Start thread	`Thread(target=task).start()`

2. Collections comparison

Collection	Syntax	Ordered?	Mutable?	Duplicates?	Access style
List	`[1, 2, 3]`	Yes	Yes	Allowed	Index
Tuple	`(1, 2, 3)`	Yes	No	Allowed	Index
Set	`{1, 2, 3}`	No	Yes	Removed	Membership / iteration
Dictionary	`{"a": 1}`	Insertion-ordered in modern Python; transcript focuses on	Yes	Keys unique	Key

Collection	Syntax	Ordered?	Mutable?	Duplicates?	Access style
		key lookup			
Array	`array('i', [1, 2])`	Sequence	Yes	Allowed	Index

3. Operators recap

Type	Operators
Arithmetic	`+ - * / // % **`
Assignment	`=` `+=` (and related augmented assignments)
Relational	`< > <= >= == !=`
Logical	`and or not`
Set operations	`- | & ^`

4. Function concepts recap

Concept	Meaning
Parameter	Name in the function definition.
Argument	Actual value supplied when calling the function.
Default argument	Value used when caller omits that parameter.
`*args`	Collects additional positional arguments into a tuple.
Keyword argument	Supplies a value by parameter name.
`return`	Sends a value back to caller.
Higher-order function	Accepts/returns another function.
Lambda	Short anonymous one-expression function.
`filter`	Keeps items whose test returns True.
`map`	Transforms each item.
`reduce`	Combines items into one accumulated result.
Inner function	Function defined inside another function.
Decorator	Wraps a function to add behaviour.

5. OOP concepts recap

Concept	Meaning
Class	Blueprint for objects.
Object	Instance created from a class.
Attribute	Data/property associated with an object/class.
Method	Function defined in a class.
`self`	Current object reference in instance methods.
`__init__`	Initialises an already-created object.
`__new__`	Creates/returns a new instance.
Inheritance	Child class reuses parent class features.
MRO	Order Python follows to find methods.
`super()`	Accesses the next class in the MRO.
Polymorphism	One operation/interface, multiple behaviours.
Duck typing	Required behaviour matters more than exact class.

Concept	Meaning
Operator overloading	Special methods define operators for custom objects.
Overriding	Child supplies its own implementation of a parent method.
Abstraction	Hide implementation and expose a common interface.

6. Error-handling recap

- **Syntax error:** Python cannot parse the statement.
- **Logical error:** Code runs but result/behaviour is wrong.
- **Runtime exception:** Failure occurs while the program executes.
- Use a specific `except` clause for expected errors.
- Use `except Exception as e` for a general fallback when appropriate.
- Use `finally` for cleanup that must happen whether execution succeeds or fails.

7. Common beginner mistakes highlighted by the transcript

104. Forgetting to add Python to PATH or not reopening the terminal after installation.
105. Saving source as `.txt` instead of `.py`.
106. Typing code directly into Command Prompt instead of the Python prompt.
107. Forgetting quotes around string literals.
108. Mismatching the opening/closing quote in a string.
109. Using `^` for exponentiation instead of `**`.
110. Expecting `/` to return an integer rather than a float.
111. Confusing `/`, `//`, and `%`.
112. Forgetting that a string and tuple are immutable while a list is mutable.
113. Treating a list assignment such as `b = a` as a copy (it is another reference).
114. Using `{}` for an empty set (it creates an empty dictionary).
115. Using `=` when comparison needs `==`.
116. Forgetting the colon or consistent indentation after a control statement.
117. Forgetting to increment a `while` loop counter.
118. Forgetting that `range()` excludes its stop value.
119. Confusing `break` (exit loop) with `continue` (skip to next iteration).
120. Forgetting to call a function after defining it.
121. Forgetting that a function without `return` returns `None`.
122. Calling an inner function from outside its scope without returning it.
123. Confusing passing a function (`square`) with calling it (`square()`).
124. Forgetting to import a module/member before using it.
125. Ignoring the current working directory when running/importing files.
126. Forgetting the base case in recursion.
127. Confusing local variables with global variables.
128. Creating a child constructor without calling the parent constructor when parent initialisation is needed.
129. Assuming a child always uses the parent method even when it overrides it.
130. Forgetting to implement an abstract method in a concrete child class.
131. Handling only one exception when several input/runtime failures are possible.
132. Putting resource cleanup only in `except` instead of `finally`.
133. Assuming threads always speed up CPU-heavy Python code; consider the GIL and multiprocessing.

8. Practice checklist based on the lesson

Use this checklist as a revision tracker:

- [ ] Explain Python as high-level, general-purpose, and multi-paradigm.
- [ ] Install Python and verify the version in the terminal.
- [ ] Create and run a `.py` file from the terminal and VS Code/PyCharm.
- [ ] Use the Python prompt/IDLE for quick calculations.
- [ ] Perform arithmetic, powers, quotient, and remainder operations.
- [ ] Create strings, use quotes/escapes, index, slice, and explain immutability.

- [ ] Create and modify lists; use common list methods.
- [ ] Create tuples, unpack them, and explain immutability.
- [ ] Create sets and perform union/intersection/difference/symmetric difference.
- [ ] Create dictionaries, use get, pop, and nested key lookup.
- [ ] Explain object identity, references, and garbage collection.
- [ ] Identify and convert Python data types.
- [ ] Use arithmetic, assignment, relational, and logical operators.
- [ ] Define functions, pass parameters/arguments, and return values.
- [ ] Import and use a module such as math.
- [ ] Swap two variables using a temporary variable and tuple unpacking.
- [ ] Read and convert user input.
- [ ] Write if, else, nested if, elif, and match statements.
- [ ] Use while and for loops; trace their iterations.
- [ ] Demonstrate break and continue.
- [ ] Create typed arrays and use array methods/copying techniques.
- [ ] Use default arguments and *args.
- [ ] Explain local/global scope and use global correctly.
- [ ] Calculate factorial using a loop and recursion.
- [ ] Pass functions to higher-order functions; write a lambda.
- [ ] Use filter, map, and reduce in a simple data pipeline.
- [ ] Define an inner function and return a function.
- [ ] Write a decorator and a generic wrapper with *args, **kwargs.
- [ ] Create and import modules from a package.
- [ ] Explain if __name__ == "__main__".
- [ ] Explain class, object, attributes, methods, and self.
- [ ] Demonstrate __init__ and __new__.
- [ ] Compare instance, class, and static methods.
- [ ] Explain inheritance, MRO, and super().
- [ ] Demonstrate duck typing, operator overloading, and overriding.
- [ ] Create an abstract class and implement its abstract method.
- [ ] Handle ZeroDivisionError, ValueError, and a general exception.
- [ ] Use finally for resource cleanup.
- [ ] Explain when to use threads vs processes.

FINAL ONE-MINUTE RECALL

Python basics: readable, general-purpose, multi-paradigm; source code is executed through the Python runtime.
Data: numbers, strings, Booleans, None, list, tuple, set, dictionary, range, and typed array.
Logic: operators + conditions + loops control program flow.
Functions: reusable code; arguments go in, return sends a result out.
Functional tools: functions can be passed/returned; lambda, filter, map, reduce, inner functions, decorators.
Modules: .py files are modules; folders group modules into packages; __name__ distinguishes direct execution from import.
OOP: class blueprint -> object instance; methods and attributes; inheritance, MRO, polymorphism, overloading, overriding, abstraction.
Reliability: distinguish syntax, logic, and runtime errors; handle exceptions and always clean up resources.
Concurrency: threads are useful for I/O-bound work; CPU-bound work may need multiprocessing because of the GIL.